

Acknowledgement

We would like to express our sincere gratitude to all the people who have supported us during the first phase of this project. This report is the result of the guidance and encouragement we received from our faculty members, family and friends.

We are thankful to our internal guide, **Prof. Dr. Nitinkumar Pandya**, for the valuable suggestions and continuous support throughout the development of this project. The regular reviews and feedback helped us stay on track and improve the quality of our work.

We are also grateful to the Head of the Department of Computer Engineering, **Dr. Ashishkumar Patel** and the institute principal, **Dr. Gargi Rajpara** for providing the necessary infrastructure and a healthy academic environment for carrying out this project.

Dhruv Prajapati

[23BECE30293]

Khush Patel

[23BECE30219]

Rutul Arya

[23BECE30006]

Prince Parmar

[23BECE30158]

Abstract

The **Digital Memory Search Engine**, referred to as Memory Engine, is an AI-assisted personal knowledge management system. The aim of the project is to provide a single platform where users can store different types of personal content such as PDF documents, DOCX files, plain text notes, images and links to YouTube videos, and later retrieve the required information using a keyword search or an AI assistant.

The current implementation covers user registration and login using JSON Web Tokens, a dashboard that shows basic statistics related to user activity, a memory management module for uploading, previewing, downloading and deleting stored memories, a keyword based search module with filtering and page level preview, and an AI assistant that answers questions from the contents of the stored memories.

The AI assistant is configured to answer only from the data that is available inside the uploaded memories. It supports basic greetings and provides source document references along with page numbers wherever applicable. Image based question answering and answering questions about YouTube videos are features that are currently under development.

This report presents the work carried out during Project Presentation-1. About sixty to seventy percent of the planned functionality has been implemented, and the remaining modules, including full semantic search over images and video content, are scheduled for the next phase. The technologies used in the project include React, HTML, CSS, JavaScript, Tailwind CSS and TypeScript on the front end, Node.js with Express.js for the backend, MongoDB for application data, Pinecone as the vector database, JWT for authentication, and the Mistral API along with Mistral embeddings for the AI assistant.

Table of Contents

Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Aims and Objective of the Work	1
1.3 Brief Literature Review	2
1.4 Problem Definition	2
1.5 Plan of Work	3
Chapter 2 Technology and Literature Review	4
2.1 Frontend Technologies	4
2.2 Backend Technologies	4
2.3 Authentication	4
2.4 Databases	5
2.5 Artificial Intelligence	5
Chapter 3 System Requirements Study	6
3.1 User Characteristics	6
3.2 Hardware and Software Requirements	6
3.3 Assumptions and Dependencies	7
Chapter 4 System Diagrams	8
4.1 System Architecture	8
4.2 Data Flow Diagram Level - 0	9
4.3 Data Flow Diagram Level - 1	10
4.4 Use Case Diagram	11
4.5 Activity Diagram	12
4.6 Sequence Diagram	14

4.7 E-R Diagram	15
Chapter 5 Data Dictionary	16
5.1 Data Dictionary Overview.....	16
5.2 Entity Attributes	16
Chapter 6 Result, Discussion and Conclusion	22
6.1 Implemented Features	22
6.2 Discussion	22
6.3 Current Limitations	23
6.4 Workflow of the System	23
6.5 Conclusion	24
6.6 Future Enhancements	24
Chapter 7 References	25

List of Figures

Figure 4.1	System Architecture -----	9
Figure 4.2	Data Flow Diagram – Level 0 -----	10
Figure 4.3	Data Flow Diagram – Level 1 -----	11
Figure 4.4	Use Case Diagram-----	12
Figure 4.5	Activity Diagram -----	13
Figure 4.6	Sequence Diagram -----	14
Figure 4.7	E-R Diagram -----	15

List of Tables

Table 1	Current Project Module Status -----	3
Table 2	Hardware Requirements-----	6
Table 3	Software Stack-----	6
Table 4	User Entity -----	16
Table 5	Memory Entity-----	17
Table 6	Document Entity -----	18
Table 7	Note Entity -----	18
Table 8	YouTube Link Entity -----	19
Table 9	Search History Entity -----	20
Table 10	AI Chat History Entity-----	21

Chapter 1

Introduction

1.1 Introduction

In recent years, the amount of personal digital content produced by a single user has grown rapidly. Students, working professionals and researchers handle hundreds of files every month in the form of PDFs, Word documents, notes, images and web references. Searching through this content in a fast and reliable way is not always simple, because the information is scattered across different applications and file types.

The Digital Memory Search Engine, also called Memory Engine, is an attempt to solve this problem by providing a single platform for storing and retrieving personal information. The system allows a user to upload documents, notes, images and YouTube links into a personal memory store. Once the content is stored, the user can either search through it using keywords or ask questions to an AI assistant. The assistant answers only from the data present in the stored memories and shows references to the source document along with the page number wherever applicable.

This report covers the work completed in the first presentation stage of the project. The implementation described here represents roughly sixty to seventy percent of the total planned scope. The remaining modules will be completed in the next phase.

1.2 Aims and Objective of the Work

The aim of this project is to design and develop a personal knowledge management system that allows users to store different types of content and retrieve relevant information quickly. The following objectives have been kept in mind during the development of the current version of the system:

- To provide secure user registration and login so that each user has a private memory space.
- To allow users to upload documents in multiple formats including PDF, DOCX, plain text, image files and YouTube links.
- To provide a keyword-based search module that returns matching documents and page-level previews.

- To provide an AI assistant that can answer natural language questions using only the user's stored memories.
- To show source references such as document name and page number for every answer produced by the AI assistant.
- To display basic usage statistics to the user through a dashboard view.

1.3 Brief Literature Review

Personal knowledge management has been an active area of research for several years. Tools such as note taking applications, document managers and bookmark managers have existed for a long time. They usually work with a single type of content and provide only basic search facilities.

Recent developments in large language models and vector databases have made it possible to build retrieval based question answering systems. In such systems, user content is first converted into embeddings and stored in a vector database. When the user asks a question, the most relevant pieces of stored content are retrieved and passed to the language model, which then produces an answer that is grounded in the user's own data.

Memory Engine follows the same general idea but keeps the design simple by combining a traditional keyword search with a vector based retrieval step. The keyword search helps the user in locating a known term quickly, while the retrieval step supports the AI assistant in finding the most relevant context for a user query.

1.4 Problem Definition

Most users maintain personal content in separate folders and applications. There is no single place where a user can store documents, images, notes and video references together and then ask questions about them in natural language.

Existing retrieval systems also rely heavily on exact keyword matching. When the user does not remember the exact word or phrase used in the document, a simple keyword search may not return a useful result. Likewise, general purpose AI chat tools answer questions from knowledge available on the public internet but do not have access to a user's private collection of files.

The problem that this project tries to solve is therefore the following: how to provide a personal, private and easy to use system that allows a user to store different types of content and then search or ask questions about that content through a single interface.

1.5 Plan of Work

The work for the project has been divided into a number of phases. In the first phase, the basic frontend and backend structure was set up, authentication was implemented and the memory upload and download modules were completed. A basic keyword search was also implemented along with page level preview of search results.

In the second phase, the AI assistant was integrated. The assistant retrieves relevant memory chunks using vector search and uses a Mistral based language model to generate the final answer. Source referencing and basic conversational responses were added during this phase.

The third phase, which is planned for the next semester, will cover improvements in image based retrieval, YouTube video transcript processing, shared access, tag editing and renaming of stored memories. A more advanced data dictionary and the complete set of system diagrams will also be prepared during this phase.

The following table summarises the current status of work on a module by module basis for the first presentation.

Module	Status	Phase
Authentication	Completed	Phase 1
Dashboard	Completed	Phase 1
Memory Upload and Preview	Completed	Phase 1
Keyword Search	Completed	Phase 1
AI Assistant (Text based)	Completed	Phase 2
Image based Retrieval	Pending	Phase 3
YouTube based Retrieval	Pending	Phase 3
Rename, Share, Edit Tags	Pending	Phase 3

Table 1: Current Project Module Status

Chapter 2

Technology and Literature Review

This chapter describes the technologies used in the project and the related background work that has influenced the design of the system. Only technologies that are part of the current implementation are described here.

2.1 Frontend Technologies

The user interface of Memory Engine is built using React. React allows the application to be developed as a set of reusable components and makes it easier to manage complex views such as the dashboard, the upload page and the search results page. The layout and styling are handled using HTML, CSS, JavaScript and Tailwind CSS. Tailwind provides a utility based approach to styling that keeps the design consistent and reduces the amount of repeated CSS code.

TypeScript is used in the API layer of the project. TypeScript adds static typing on top of JavaScript, which makes the request and response handling code easier to understand and reduces the number of common runtime errors caused by incorrect data structures.

2.2 Backend Technologies

The backend of Memory Engine is implemented in Node.js using Express.js. Express.js is a lightweight web framework that supports the quick creation of REST based APIs. The backend exposes endpoints for user authentication, memory upload, memory listing, search and chat with the AI assistant.

All communication between the frontend and the backend is done over a REST API. Each request from the frontend carries a JWT based access token in the headers so that the backend can identify the user before processing the request.

2.3 Authentication

User authentication is done using JSON Web Tokens. When a user registers or logs in, the backend verifies the credentials and returns a signed JWT. This token is then stored on the client side and sent with every subsequent request. JWT allows the backend to remain stateless while still being able to recognise the logged in user.

2.4 Databases

Two databases are used in the project. MongoDB is used as the primary database for storing user accounts, uploaded memory metadata, file references and chat logs. MongoDB is chosen because it stores data in flexible JSON like documents, which is convenient when working with a JavaScript and TypeScript based stack.

Pinecone is used as the vector database. When a memory is uploaded, its text content is split into smaller chunks and passed through an embedding model. Each chunk is converted into a numerical vector and stored in Pinecone along with a reference to its source document and page. During a question answering session, the AI assistant searches Pinecone for the chunks that are most similar to the user query.

2.5 Artificial Intelligence

The AI assistant is powered by the Mistral API along with Mistral embeddings. Mistral embeddings are used to convert text content of uploaded memories into vector form. The same kind of vector is produced for the user query. The assistant then uses the Mistral language model to read the top ranked chunks retrieved from Pinecone and produce a final answer in natural language.

The assistant is configured to answer only from the retrieved chunks. It does not fall back on general world knowledge. This keeps the answers grounded in the user's own data and reduces the chance of generating information that is not present in the uploaded memories.

Chapter 3

System Requirements Study

This chapter discusses the system requirements for the project. It covers the characteristics of the intended users, the hardware and software environment needed to run the system and the assumptions and dependencies that have been made during the current phase of development.

3.1 User Characteristics

The intended users of Memory Engine are undergraduate and postgraduate students, working professionals and researchers who regularly deal with large amounts of personal study or work material. Users are expected to be familiar with basic operations such as uploading files, searching the internet and using a browser based application.

The application is designed to be simple enough for a first time user to upload a document and perform a search without any training. At the same time, it provides an AI based assistant that can support more advanced natural language queries once the user becomes comfortable with the system.

3.2 Hardware and Software Requirements

Memory Engine is a web based application. The following table lists the recommended hardware configuration for a development machine and a typical user machine.

Component	Developer Machine	User Machine
Processor	Intel i5 or higher	Intel i3 or higher
RAM	8 GB or more	4 GB or more
Storage	256 GB SSD	Any modern device
Network	Stable broadband	Stable internet
Browser	Chrome / Firefox (latest)	Chrome / Firefox (latest)

Table 2: Hardware Requirements

The following table lists the software technologies that are used in the project.

Layer	Technology	Purpose
Frontend	React, HTML, CSS, JavaScript, Tailwind CSS	User interface
API Layer	TypeScript	Typed request and response handling

Backend	Node.js, Express.js	REST API and business logic
Authentication	JWT	Secure user sessions
Database	MongoDB	User accounts and metadata storage
Vector Database	Pinecone	Similarity search over embeddings
AI Model	Mistral API, Mistral Embeddings	Question answering

Table 3: Software Stack

3.3 Assumptions and Dependencies

The current version of the project is developed and tested on Windows 10 and Windows 11 with Node.js version 18 or higher. The following assumptions have been made during development:

- The user has a stable internet connection as the AI assistant depends on the Mistral API and Pinecone cloud service.
- The user provides text based documents such as PDF or DOCX for best results with the AI assistant.
- Each registered user has a unique email address that will be used as the login identifier.
- The browser used by the user supports modern JavaScript features required by React.

The project depends on the availability of the Mistral API and Pinecone vector database. Any change in the pricing or service level of these external services may affect the working of the AI assistant. The project also depends on standard NPM packages for authentication, file parsing and embedding generation.

Chapter 4

System Diagrams

This chapter briefly presents the system design aspects of the project.

4.1 System Architecture

The System Architecture illustrates the overall structure of the **Digital Memory Search Engine** and the interaction between its major components. It shows how the frontend, backend, databases, and AI services work together to provide secure storage, semantic search, and AI-assisted document retrieval.

- The **React-based frontend** provides the user interface for login, memory management, search, AI assistant, and settings.
- The **Node.js + Express.js backend** processes user requests, manages authentication, and communicates with all services.
- **MongoDB** stores user accounts, document metadata, notes, and other application data.
- **Pinecone Vector Database** stores document embeddings for semantic search.
- **Mistral Embedding API** converts uploaded document content into vector embeddings.
- **Mistral API** generates AI responses using the relevant context retrieved from the stored memories.

Digital Memory Search Engine

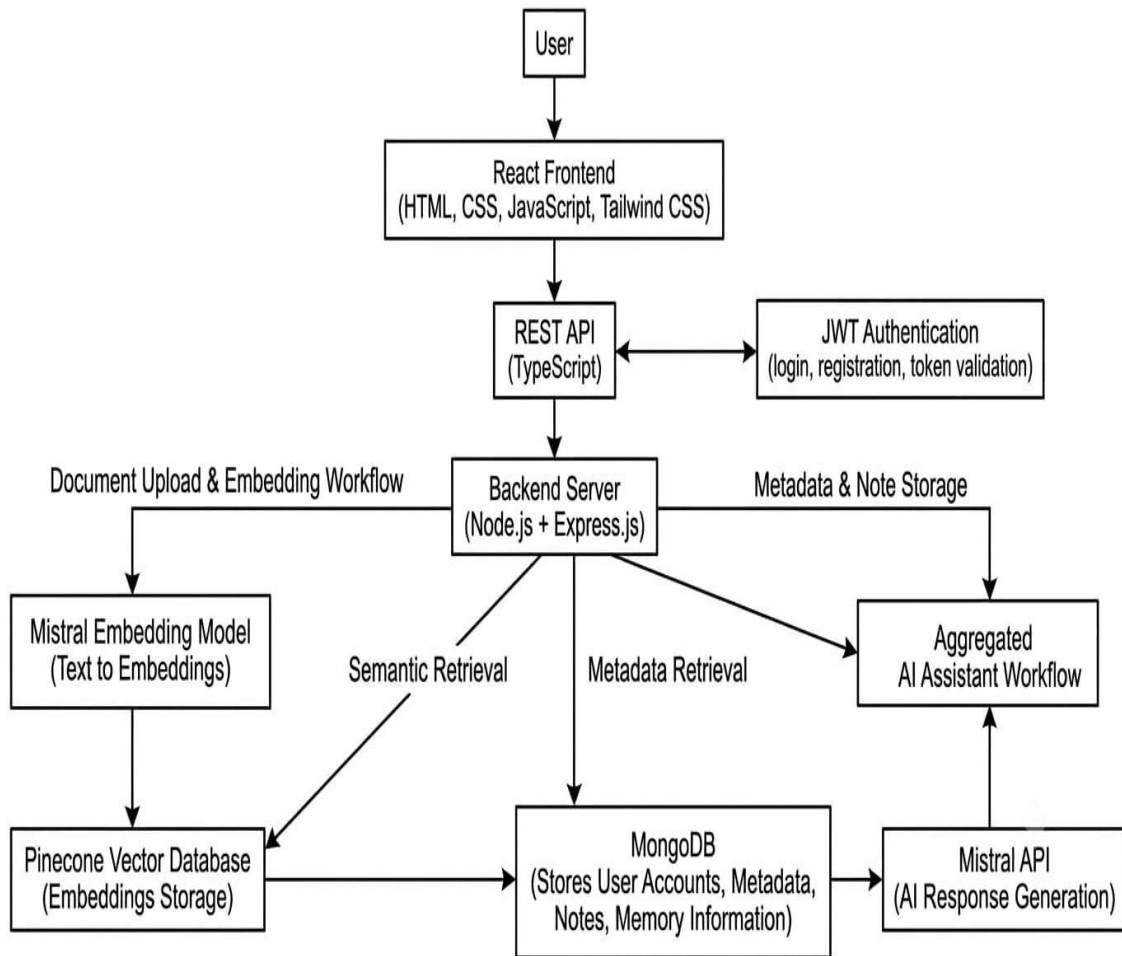


Figure 4.1: System Architecture Diagram

4.2 Data Flow Diagram – Level 0

The Level 0 Data Flow Diagram (Context Diagram) provides a high-level view of the **Digital Memory Search Engine**. It shows how the user interacts with the system and how data is exchanged with **MongoDB** and the **Pinecone Vector Database**.

- The user registers, logs in, uploads memories, searches documents, and asks questions to the AI assistant.
- **MongoDB** stores user information and memory metadata.
- **Pinecone Vector Database** stores document embeddings for semantic search.
- The system retrieves relevant information and returns search results or AI responses to the user.

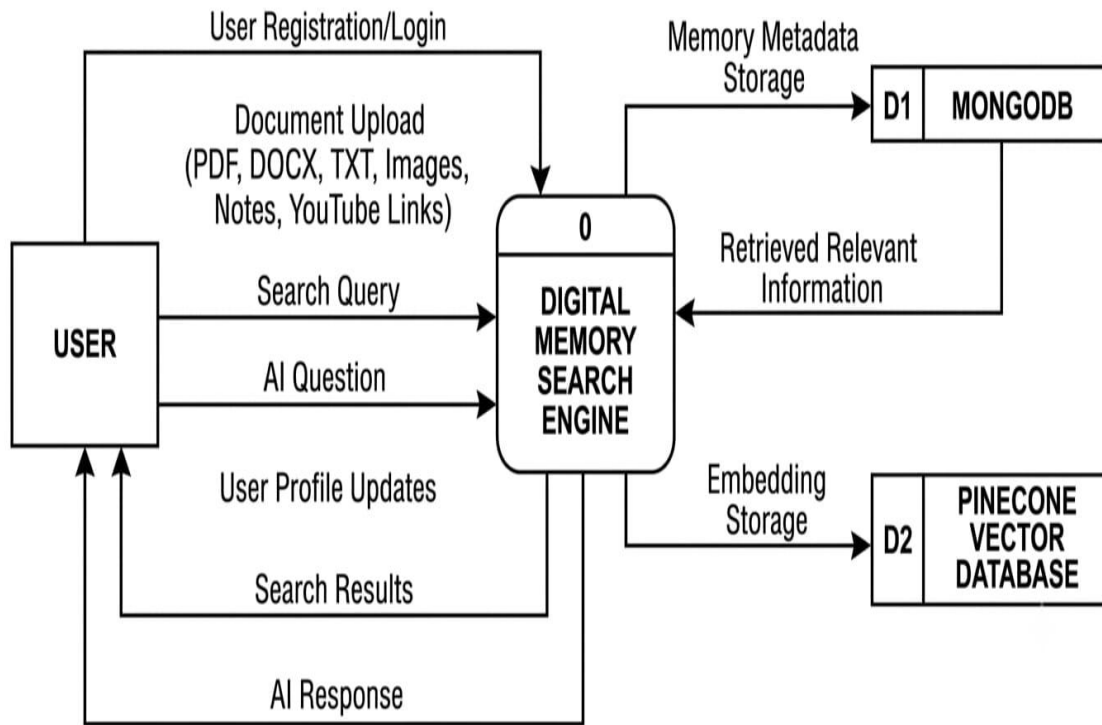


Figure 4.2: Data Flow Diagram – Level 0

4.3 Data Flow Diagram – Level 1

The Level 1 Data Flow Diagram provides a detailed view of the major functional modules of the Digital Memory Search Engine and the flow of data between them.

- **User Authentication:** Verifies user credentials and manages secure login using JWT authentication.
- **Memory Management:** Handles uploading, storing, viewing, and deleting memories while maintaining metadata in MongoDB.
- **Search Module:** Processes user search queries and retrieves relevant information using the Pinecone Vector Database.
- **AI Assistant:** Retrieves relevant context from stored memories and generates responses using the Mistral API.
- **Settings:** Allows users to update and manage their profile information.

Digital Memory Search Engine

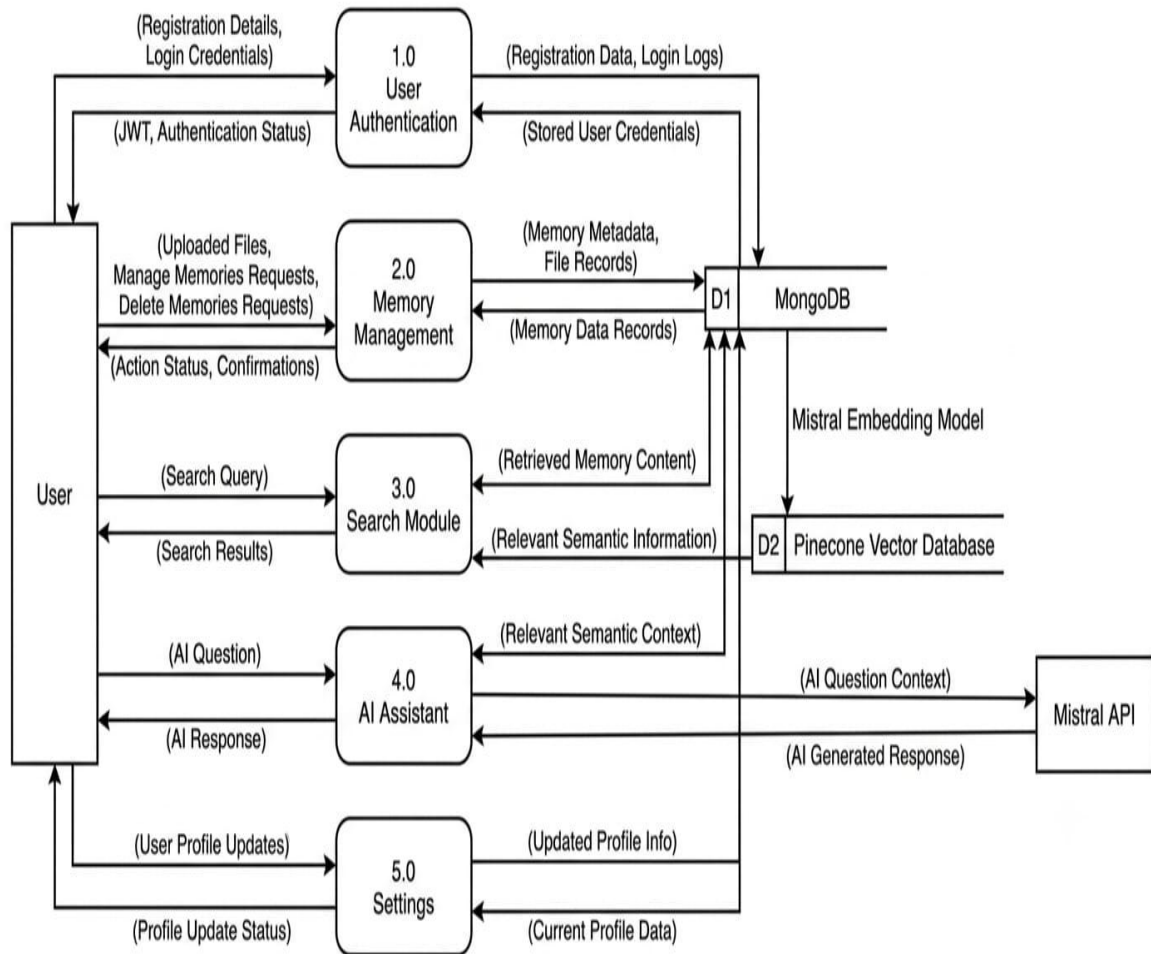


Figure 4.3: Data Flow Diagram – Level 1

4.4 Use case Diagram

The Use Case Diagram illustrates the interactions between the user and the Digital Memory Search Engine. It represents the main functionalities available to the user, including authentication, memory management, search, AI assistance, and profile management.

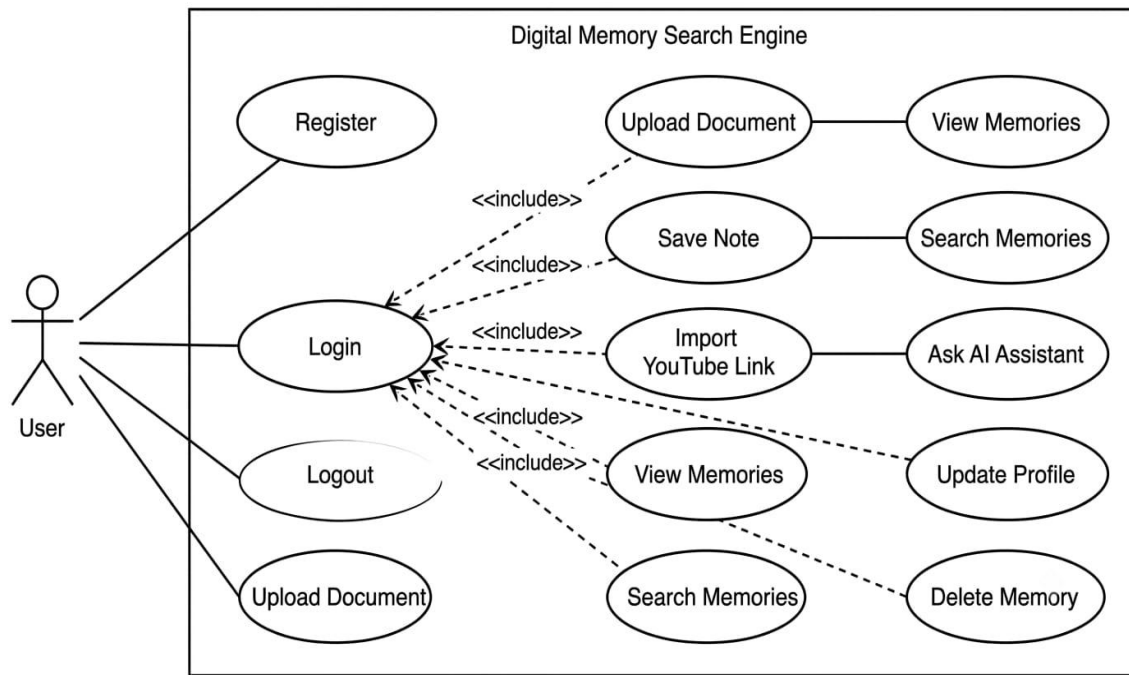


Figure 4.4: Use Case Diagram

4.5 Activity Diagram

The Activity Diagram illustrates the workflow of the Digital Memory Search Engine from user login to performing different operations. It shows the sequence of activities such as uploading memories, searching documents, interacting with the AI Assistant, managing profile settings, and logging out of the system.

Digital Memory Search Engine

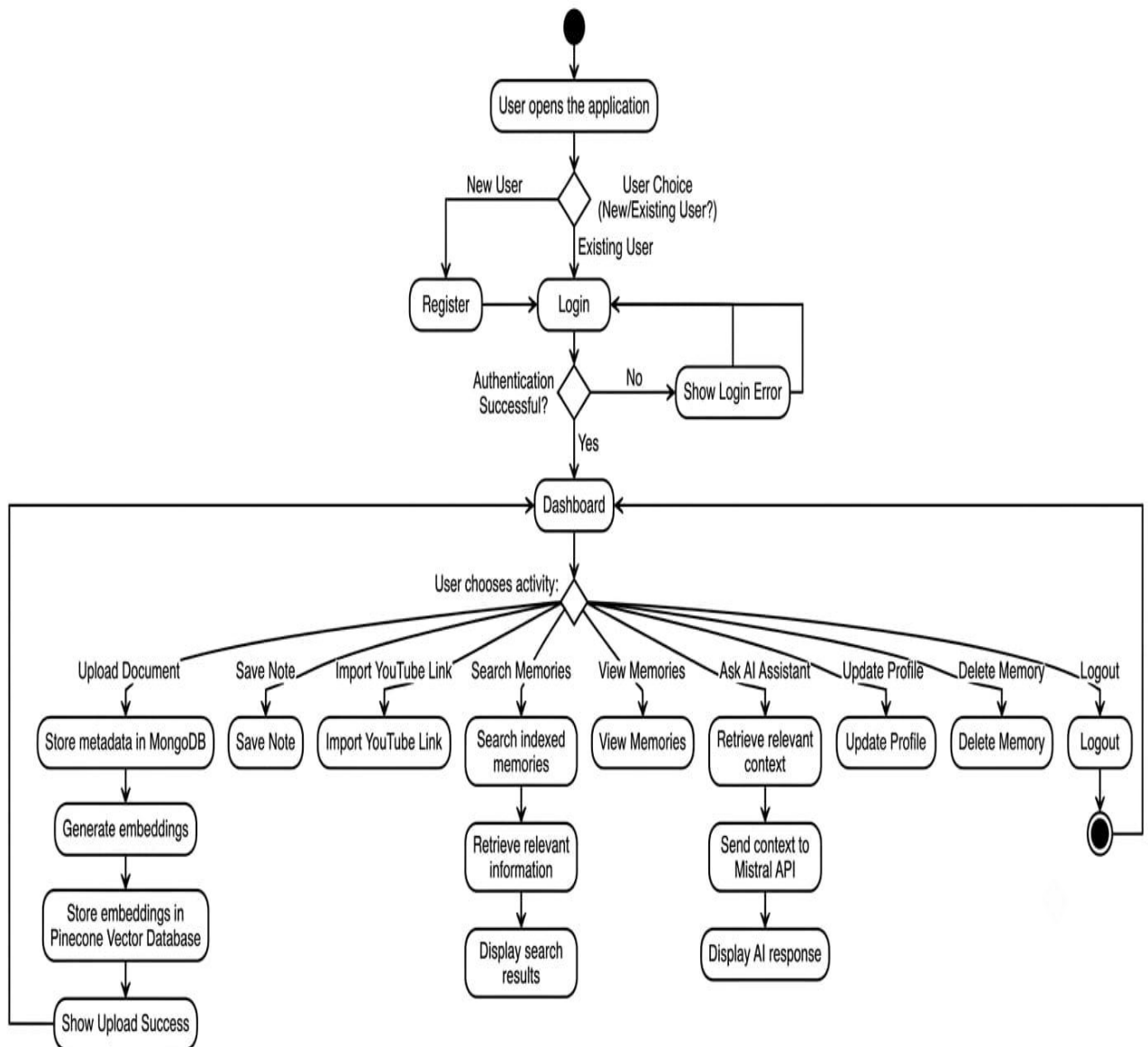


Figure 4.5: Activity Diagram

4.6 Sequence Diagram

The Sequence Diagram illustrates the interaction between the user, frontend, backend, databases, and AI services during system operations. It shows the sequence of messages exchanged for user authentication, document upload, semantic search, and AI-assisted response generation.

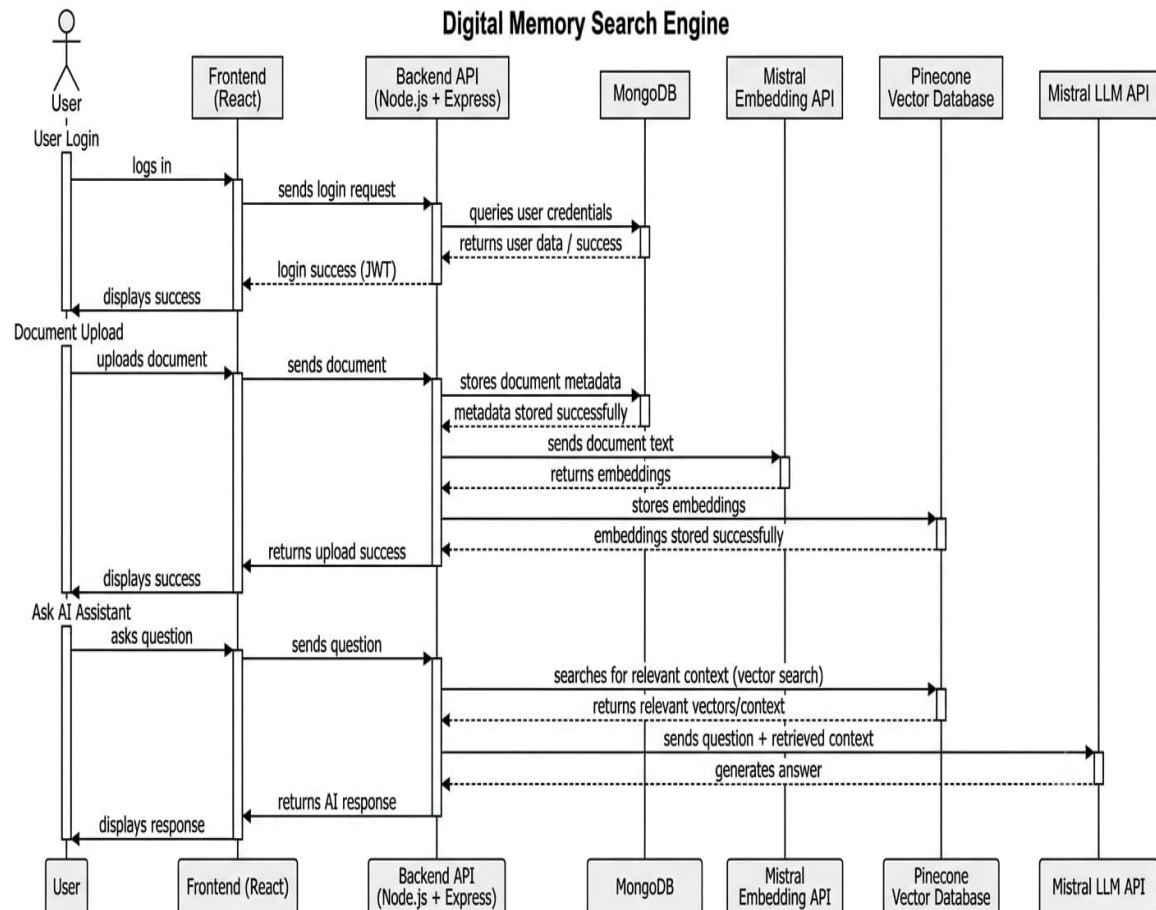


Figure 4.6: Sequence Diagram

4.7 E-R Diagram

The Entity Relationship (E-R) Diagram represents the database structure of the Digital Memory Search Engine. It illustrates the relationships between the main entities such as User, Memory, Document, Note, Search History, AI Chat History, and YouTube Link, along with their associated attributes.

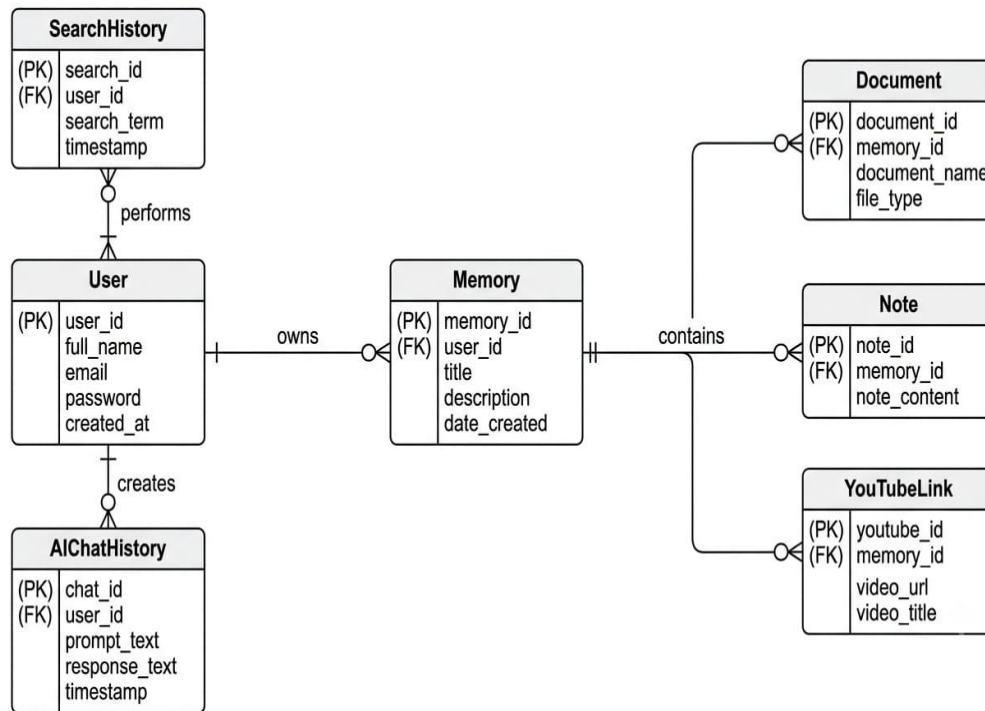


Figure 4.7: E-R Diagram

Chapter 5

Data Dictionary

5.1 Data Dictionary Overview

The Data Dictionary defines the database entities and their attributes used in the Digital Memory Search Engine. It provides details such as attribute names, data types, descriptions, and key constraints for each entity.

5.2 Entity Attributes

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
user_id	Integer	Unique identifier for each user.	10	Primary key, Not Null	Positive Integer
full_name	Varchar	Email address used for login.	100	Not Null	Alphabetical characters
email	Varchar	Email address used for login.	100	Unique, Not Null	<u>user@example.com</u>
password	Varchar	Encrypted password.	255	Not Null	Encrypted text
created_at	Timestamp	Account creation and time.	-	Not Null	YYYY-MM-DD HH:MM:SS

Table 4: User Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
memory_id	Integer	Unique identifier for each memory.	10	Primary Key, Not Null	Positive Integer
user_id	Integer	References the owner of the memory.	10	Foreign Key, Not Null	Existing User ID
title	Varchar	Title of the memory.	255	Not Null	Text
description	Text	Description or summary of the memory.	—	Optional	Text
date_created	Timestamp	Date and time when the memory was created.	—	Not Null	YYYY-MM-DD HH:MM:SS

Table 5: Memory Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
document_id	Integer	Unique identifier for each document.	10	Primary Key, Not Null	Positive Integer
memory_id	Integer	References the associated memory.	10	Foreign Key, Not Null	Existing Memory ID
document_name	Varchar	Name of the uploaded document.	255	Not Null	Text
file_type	Varchar	Type of uploaded file (e.g., PDF, DOCX, TXT).	50	Not Null	PDF, DOCX, TXT, etc.

Table 6: Document Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
note_id	Integer	Unique identifier for each note.	10	Primary Key, Not Null	Positive Integer
memory_id	Integer	References the associated memory.	10	Foreign Key, Not Null	Existing Memory ID

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
note_content	Text	Content of the saved note.	—	Not Null	Text

Table 7: Note Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
youtube_id	Integer	Unique identifier for each YouTube link.	10	Primary Key, Not Null	Positive Integer
memory_id	Integer	References the associated memory.	10	Foreign Key, Not Null	Existing Memory ID
video_url	Varchar	URL of the YouTube video.	255	Not Null	Valid YouTube URL
video_title	Varchar	Title of the YouTube video.	255	Optional	Text

Table 8: YouTube Link Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
search_id	Integer	Unique identifier for each search record.	10	Primary Key, Not Null	Positive Integer
user_id	Integer	References the user who performed the search.	10	Foreign Key, Not Null	Existing User ID
query	Varchar	Search keyword or phrase entered by the user.	255	Not Null	Text
searched_at	Timestamp	Date and time when the search was performed.	—	Not Null	YYYY-MM-DD HH:MM:SS

Table 9: Search History Entity

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
chat_id	Integer	Unique identifier for each chat record.	10	Primary Key, Not Null	Positive Integer

Attribute Name	Data Type	Description	Size	Constraints	Range/Valid Values
user_id	Integer	References the user who initiated the chat.	10	Foreign Key, Not Null	Existing User ID
prompt	Text	User's question submitted to the AI Assistant.	—	Not Null	Text
response	Text	AI-generated response returned to the user.	—	Not Null	Text
timestamp	Timestamp	Date and time when the chat was recorded.	—	Not Null	YYYY-MM-DD HH:MM:SS

Table 10: AI Chat History Entity

The data dictionary provides a structured description of the entities and attributes used in the Digital Memory Search Engine. It serves as a reference for the database design and implementation of the system.

Chapter 6

Result, Discussion and Conclusion

6.1 Implemented Features

The following features have been implemented and tested as part of the first presentation of the project.

- User Registration and Login with secure password storage and JWT based authentication.
- A dashboard that displays total searches, today's searches, average number of search results per query and total indexed documents.
- Memory Management module that supports uploading, previewing, downloading and deleting memories.
- Support for uploading PDF, DOCX and TXT files, image files, plain text notes and YouTube links.
- Keyword Search module with filtering, page level preview and a button to open the page containing the matched text.
- AI Assistant that answers questions using the contents of the uploaded memories, shows source document references and page numbers wherever applicable.
- Basic conversational responses to greetings such as Hi, Hello, Good Morning, Good Night and How are you.
- Settings page that allows the user to view and update basic account details.

6.2 Discussion

The first phase of the project has shown that combining a keyword search with vector based retrieval is a practical way to support both direct and natural language queries on a personal knowledge base. Users can quickly locate a known term using the keyword search and, when they do not remember the exact wording, they can ask the assistant in natural language and still receive a useful answer with a source reference.

A useful observation during testing was that the AI assistant gives more accurate answers when the uploaded documents are text heavy. PDFs that contain a lot of scanned images may not be read as well, although PDF text extraction is in place for standard

PDFs. Image based questions and understanding of YouTube video content are still areas that need improvement and are scheduled for the next phase.

Some of the UI items such as Rename, Share and Edit Tags for stored memories have been kept on the screen but are marked as Coming Soon. This was done so that the user can see the planned scope of the application from an early stage, while making it clear that these actions are not yet active.

6.3 Current Limitations

Because the project is still in its first presentation stage, a few features are partly or fully pending. The main limitations of the current implementation are listed below.

- The AI assistant is not yet able to answer questions meaningfully based on the contents of uploaded images.
- The AI assistant cannot retrieve information from the transcripts of the linked YouTube videos.
- Rename, Share and Edit Tags options are visible in the UI but are marked as Coming Soon and are not active.

6.4 Workflow of the System

The basic workflow of Memory Engine as implemented at present is as follows. The user first registers and logs in to the system. After login, the user can upload memories in any of the supported formats. The system stores the metadata of each memory in MongoDB and, for text based memories, stores the embeddings of the text content in Pinecone.

When the user wants to find something, there are two options. The user may either use the keyword search module, which returns documents and page level previews for matching content, or ask a question to the AI assistant. In the second case, the system first retrieves the most relevant memory chunks from Pinecone and then passes them to the Mistral model which produces a short answer together with the source references.

The overall workflow can be represented in a simple step wise manner:

- Step 1: User logs in to the system.
- Step 2: User uploads one or more memories.

- Step 3: The system stores the memory data.
- Step 4: User performs a keyword search or asks the AI assistant.
- Step 5: The system retrieves relevant information.
- Step 6: The relevant answer or matching pages are shown to the user.

6.5 Conclusion

The work completed in the first presentation covers all the basic modules of Memory Engine. A user can register, upload personal content in different formats and use either the keyword search or the AI assistant to retrieve information from those memories. The system is already usable in a day to day scenario for text based personal documents.

The second phase of the project will focus on improving the retrieval of information from images and YouTube videos, adding support for renaming, sharing and tagging of memories and preparing the complete system design and data dictionary. Once these features are added, Memory Engine will provide a more complete personal knowledge management experience.

6.6 Future Enhancements

The following enhancements are planned for the next phase of the project.

- Image based question answering using vision capable models.
- Processing and indexing of YouTube video transcripts.
- Activation of Rename, Share and Edit Tags options on stored memories.
- Improved ranking of vector search results.
- Multi device synchronisation of stored memories.

Chapter 7

References

- [1] React Documentation. <https://reactjs.org/docs/getting-started.html>
- [2] Node.js Documentation. <https://nodejs.org/en/docs/>
- [3] Express.js Documentation. <https://expressjs.com/>
- [4] MongoDB Documentation. <https://www.mongodb.com/docs/>
- [5] Pinecone Documentation. <https://docs.pinecone.io/>
- [6] Mistral AI Documentation. <https://docs.mistral.ai/>
- [7] JWT Introduction. <https://jwt.io/introduction>
- [8] Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- [9] TypeScript Documentation. <https://www.typescriptlang.org/docs/>